

Started on	Thursday, 12 June 2025, 12:58 AM
State	Finished
Completed on	Saturday, 14 June 2025, 11:22 AM
Time taken	2 days 10 hours
Marks	4.00/4.00
Grade	10.00 out of 10.00 (100%)

Viết chương trình đọc vào một **đơn đồ thị có hướng** và kiểm tra xem nó có chứa chu trình hay không.

Nhắc lại: *Chu trình là một đường đi có đỉnh đầu trùng với đỉnh cuối.*

Đầu vào (Input):

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u, v mô tả cung (u, v) .

Đầu ra (Output):

- In ra màn hình **CIRCLED** nếu đồ thị có chứa chu trình, ngược lại in ra **NO CIRCLE**
- Xem thêm ví dụ bên dưới.

For example:

Input	Result
3 3 1 3 3 2 2 1	CIRCLED
7 9 1 2 1 3 1 4 2 3 2 6 3 7 4 5 5 3 5 7	NO CIRCLE

Answer: (penalty regime: 10, 20, ... %)

```

1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4  typedef int ElementType;
5
6  //-----Graph-----
7  typedef struct{
8      int n,m;
9      int A[MAX_SIZE][MAX_SIZE];
10 }Graph;
11
12 //init graph with n vertex
13 void init_graph(Graph* pG, ElementType n){
14     pG->n = n;
15     pG->m = 0;
16
17     //cho toan bo ma tran ke bang 0
18     for(int i = 0; i <= pG->n; i++){
19         for(int j = 0; j <= pG->n; j++){
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 //add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v){
27     pG->A[u][v] = 1;
28     // pG->A[v][u] = 1;
29     pG->m++;
30 }
31
32 //-----DFS-----
33 #define WHITE 0
34 #define GRAY 1
35 #define BLACK 2
36
37 int color[MAX_SIZE];
38 int has_cycle;
```

```

39
40 int visited[MAX_SIZE];
41 int parent[MAX_SIZE];
42
43 int adj(Graph* pG, int u, int v){
44     return pG->A[u][v] > 0;
45 }
46 void DFS (Graph* pG, int s){
47     //1. to mau
48     color[s] = GRAY;
49
50     // visited[s] = 1;
51     // printf("%d \n", s);
52     // parent[s] = p;
53     for(int u = 1; u <= pG->n; u++){
54         if(adj(pG,s,u)){
55             if(color[u] == WHITE){
56                 DFS(pG,u);
57             } else if(color[u] == GRAY){
58                 has_cycle = 1;
59             }
60         }
61     }
62     color[s] = BLACK;
63 }
64
65 int main(){
66     Graph G;
67     int n, m, u, v, e;
68     scanf("%d%d", &n, &m);
69     init_graph(&G, n);
70
71     for (e = 0; e < m; e++) {
72         scanf("%d%d", &u, &v);
73         add_edge(&G, u, v);
74     }
75     has_cycle = 0;
76     for(int i = 0; i <= G.n; i++){
77         visited[i] = 0;
78         // parent[i] = -1;
79         color[i] = WHITE;
80     }
81
82     for(int i = 1; i <= G.n; i++){
83         if(!visited[i]){
84             DFS(&G,i);
85         }
86     }
87
88     if(has_cycle){
89         printf("CIRCLED");
90     }else{
91         printf("NO CIRCLE");
92     }
93
94 }

```

Debug: source code from all test runs

Run 1

```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v] = 1;
    //    pG->A[v][u] = 1;
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    //    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(color[u] == WHITE){
                DFS(pG,u);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);
    init_graph(&G, n);

    for (e = 0; e < m; e++) {

```

```
scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
has_cycle = 0;
for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
//    parent[i] = -1;
    color[i] = WHITE;
}

for(int i = 1; i <= G.n; i++){
    if(!visited[i]){
        DFS(&G,i);
    }
}

if(has_cycle){
    printf("CIRCLED");
}else{
    printf("NO CIRCLE");
}
}
```

Run 2

```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v] = 1;
    //    pG->A[v][u] = 1;
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    //    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(color[u] == WHITE){
                DFS(pG,u);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);
    init_graph(&G, n);

    for (e = 0; e < m; e++) {

```

```
        scanf("%d%d", &u, &v);
        add_edge(&G, u, v);
    }
    has_cycle = 0;
    for(int i = 0; i <= G.n; i++){
        visited[i] = 0;
//        parent[i] = -1;
        color[i] = WHITE;
    }

    for(int i = 1; i <= G.n; i++){
        if(!visited[i]){
            DFS(&G,i);
        }
    }

    if(has_cycle){
        printf("CIRCLED");
    }else{
        printf("NO CIRCLE");
    }
}
```

Run 3

```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v] = 1;
    //    pG->A[v][u] = 1;
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    //    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(color[u] == WHITE){
                DFS(pG,u);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);
    init_graph(&G, n);

    for (e = 0; e < m; e++) {

```



```
scanf("%d%d", &u, &v);
add_edge(&G, u, v);
}
has_cycle = 0;
for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
//    parent[i] = -1;
    color[i] = WHITE;
}

for(int i = 1; i <= G.n; i++){
    if(!visited[i]){
        DFS(&G,i);
    }
}

if(has_cycle){
    printf("CIRCLED");
}else{
    printf("NO CIRCLE");
}
}
```

Run 4

```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v] = 1;
    //    pG->A[v][u] = 1;
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    //    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(color[u] == WHITE){
                DFS(pG,u);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);
    init_graph(&G, n);

    for (e = 0; e < m; e++) {

```

```
scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
has_cycle = 0;
for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
//    parent[i] = -1;
    color[i] = WHITE;
}

for(int i = 1; i <= G.n; i++){
    if(!visited[i]){
        DFS(&G,i);
    }
}

if(has_cycle){
    printf("CIRCLED");
}else{
    printf("NO CIRCLE");
}
}
```

Run 5

```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v] = 1;
    //    pG->A[v][u] = 1;
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    //    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(color[u] == WHITE){
                DFS(pG,u);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);
    init_graph(&G, n);

    for (e = 0; e < m; e++) {

```

```
scanf("%d%d", &u, &v);
add_edge(&G, u, v);
}
has_cycle = 0;
for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
//    parent[i] = -1;
    color[i] = WHITE;
}

for(int i = 1; i <= G.n; i++){
    if(!visited[i]){
        DFS(&G,i);
    }
}

if(has_cycle){
    printf("CIRCLED");
}else{
    printf("NO CIRCLE");
}
}
```

Run 6

```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v] = 1;
    //    pG->A[v][u] = 1;
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    //    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(color[u] == WHITE){
                DFS(pG,u);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);
    init_graph(&G, n);

    for (e = 0; e < m; e++) {

```

```

        scanf("%d%d", &u, &v);
        add_edge(&G, u, v);
    }
    has_cycle = 0;
    for(int i = 0; i <= G.n; i++){
        visited[i] = 0;
        parent[i] = -1;
        color[i] = WHITE;
    }

    for(int i = 1; i <= G.n; i++){
        if(!visited[i]){
            DFS(&G,i);
        }
    }

    if(has_cycle){
        printf("CIRCLED");
    }else{
        printf("NO CIRCLE");
    }
}

```

	Input	Expected	Got	
✓	3 3 1 3 3 2 2 1	CIRCLED	CIRCLED	✓
✓	7 9 1 2 1 3 1 4 2 3 2 6 3 7 4 5 5 3 5 7	NO CIRCLE	NO CIRCLE	✓
✓	7 11 1 2 1 3 2 4 2 5 2 6 3 2 5 3 3 6 4 7 7 5 6 4	CIRCLED	CIRCLED	✓
✓	3 2 1 2 2 3	NO CIRCLE	NO CIRCLE	✓
✓	4 3 1 2 2 3 4 2	NO CIRCLE	NO CIRCLE	✓
✓	4 4 1 2 2 3 3 4 4 1	CIRCLED	CIRCLED	✓

Passed all tests! ✓

```

1 #include <stdio.h>
2
3 /* Khai báo CTDL Graph*/
4 #define MAX_N 100
5 typedef struct {
6     int n, m;
7     int A[MAX_N][MAX_N];
8 } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1; u <= n; u++)
14         for (int v = 1; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     //if (u != v)
21     //    pG->A[v][u] += 1;
22
23     if (pG->A[u][v] > 1)
24         printf("da cung (%d, %d)\n", u, v);
25     if (u == v)
26         printf("khuyen %d\n", u);
27
28     pG->m++;
29 }
30
31 int adjacent(Graph *pG, int u, int v) {
32     return pG->A[u][v] > 0;
33 }
34
35 #define WHITE 0
36 #define GRAY 1
37 #define BLACK 2
38
39 int color[MAX_N]; //Lưu trạng thái của các đỉnh
40 int has_circle; //Đồ thị chứa trình hay không
41
42 void DFS(Graph *pG, int u) {
43     //1. Tô màu đang duyệt cho u
44     color[u] = GRAY;
45
46     //2. Xét các đỉnh kề của u
47     for (int v = 1; v <= pG->n; v++)
48         if (adjacent(pG, u, v)) {
49             if (color[v] == WHITE) //2a. Nếu v chưa duyệt
50                 DFS(pG, v); //gọi đệ quy duyệt nó
51             else if (color[v] == GRAY) //2b. v đang duyệt
52                 has_circle = 1; //chứa chu trình
53         }
54
55     //3. Tô màu duyệt xong cho u
56     color[u] = BLACK;
57 }
58
59 int main() {
60     //1. Khai báo đồ thị G
61     Graph G;
62     //2. Đọc dữ liệu và dựng đồ thị
63     int n, m, u, v;
64     scanf("%d%d", &n, &m);
65     init_graph(&G, n);
66     for (int e = 0; e < m; e++) {
67         scanf("%d%d", &u, &v);
68         add_edge(&G, u, v);
69     }
70
71     for (int u = 1; u <= G.n; u++)
72         color[u] = WHITE;
73     //2. Khởi tạo biến has_circle
74     has_circle = 0;

```



```

77 //3. Duyệt toàn bộ đồ thị để kiểm tra chu trình
78 for (int u = 1; u <= G.n; u++)
79     if (color[u] == WHITE) //u chưa duyệt
80         DFS(&G, u); //gọi DFS(&G, u) để duyệt từ u
81 //4. Kiểm tra has_circle
82
83 if (has_circle)
84     printf("CIRCLED\n");
85 else
86     printf("NO CIRCLE\n");
87
88 return 0;
89 }
90
91

```

Correct

Marks for this submission: 1.00/1.00.

Viết chương trình đọc vào một **đơn đồ thị có hướng** và kiểm tra xem nó có chứa chu trình hay không. Nếu đồ thị có chu trình, in ra các đỉnh có trong chu trình.

Đầu vào (Input):

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u, v mô tả cung (u, v) .

Đầu ra (Output):

- Nếu đồ thị có chu trình, in ra các đỉnh có trong chu trình trên cùng 1 dòng, cách nhau 1 khoảng trắng. Đỉnh đầu và đỉnh cuối của chu trình giống nhau.
- Nếu đồ thị không chứa chu trình, in ra -1
- Nếu đồ thị có nhiều chu trình, chỉ cần in ra 1 chu trình bất kỳ.
- Xem thêm ví dụ bên dưới.

Trong ví dụ đầu tiên, ta tìm thấy chu trình $1 \rightarrow 3 \rightarrow 2 \rightarrow 1$, vì thế chương trình phải in ra:

```
1 3 2 1
```

For example:

Input	Result
3 3 1 3 3 2 2 1	Chu trình hợp lệ.
7 9 1 2 1 3 1 4 2 3 2 6 3 7 4 5 5 3 5 7	-1

Answer: (penalty regime: 10, 20, ... %)

```

1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4 typedef int ElementType;
5
6 //-----Graph-----
7 typedef struct {
8     int n, m;
9     int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 void init_graph(Graph* pG, int n) {
13     pG->n = n;
14     pG->m = 0;
15     for (int i = 0; i <= n; i++) {
16         for (int j = 0; j <= n; j++) {
17             pG->A[i][j] = 0;
18         }
19     }
20 }
21
22 void add_edge(Graph* pG, int u, int v) {
23     pG->A[u][v]++;
24     // if (u != v) pG->A[v][u]++;
25
26     pG->m++;
27 }
28
29 int adj(Graph* pG, int u, int v) {
30     return pG->A[u][v] > 0;
31 }
```

```

31 //-----Cycle-----
32 #define NO_COLOR 0
33 #define GRAY 1
34 #define BLACK 2
35
36 int color[MAX_SIZE];
37 int has_cycle;
38 int parent[MAX_SIZE];
39
40 int start, end;
41
42
43 void cycle(Graph* pG, int u) {
44     color[u] = GRAY;
45
46     for (int v = 1; v <= pG->n; v++) {
47         if (adj(pG, u, v)) {
48             if (color[v] == NO_COLOR) {
49                 parent[v] = u;
50                 cycle(pG, v);
51                 if (has_cycle) return;
52             } else if (color[v] == GRAY) {
53                 has_cycle = 1;
54                 start = v;
55                 end = u;
56                 return;
57             }
58         }
59     }
60     color[u] = BLACK;
61 }
62
63 void print_path() {
64     int path[MAX_SIZE];
65     int count = 0;
66
67     path[count++] = start;
68     int u = end;
69     while (u != start) {
70         path[count++] = u;
71         u = parent[u];
72     }
73     path[count++] = start;
74
75     for (int i = count - 1; i >= 0; i--) {
76         printf("%d ", path[i]);
77     }
78     printf("\n");
79 }
80
81 int main() {
82     int n, m;
83     scanf("%d %d", &n, &m);
84
85     Graph G;
86     init_graph(&G, n);
87
88     // Read graph
89     for (int i = 0; i < m; i++) {
90         int u, v;
91         scanf("%d %d", &u, &v);
92         add_edge(&G, u, v);
93     }
94
95     for (int i = 1; i <= n; i++) {
96         color[i] = NO_COLOR;
97         parent[i] = -1;
98     }
99
100    for (int i = 1; i <= G.n; i++) {
101        if (color[i] == NO_COLOR) {
102            start = i;
103            cycle(&G, i);
104            if (has_cycle) break;
105        }
106    }
107
108    if (has_cycle) {
109        print_path();
110    }

```

```

109     print_path();
110 } else {
111     printf("-1");
112 }
113 // for (int i = 1; i <= G.n; i++) {
114 //     printf("%d: %d\n", i, parent[i]);
115 // }
116 }
117

```

	Input	Expected	Got	
✓	3 3 1 3 3 2 2 1	Chu trình hợp lệ.	Chu trình hợp lệ.	✓
✓	7 9 1 2 1 3 1 4 2 3 2 6 3 7 4 5 5 3 5 7	-1	-1	✓
✓	7 11 1 2 1 3 2 4 2 5 2 6 3 2 5 3 3 6 4 7 7 5 6 4	Chu trình hợp lệ.	Chu trình hợp lệ.	✓
✓	3 2 1 2 2 3	-1	-1	✓
✓	4 3 1 2 2 3 4 2	-1	-1	✓
✓	4 4 1 2 2 3 3 4 4 1	Chu trình hợp lệ.	Chu trình hợp lệ.	✓

Passed all tests! ✓

Question author's solution (Python3):

```

1 #include <stdio.h>
2
3 /* Khai báo CTDL Graph*/
4 #define MAX_N 100
5 typedef struct {
6     int n, m;
7     int A[MAX_N][MAX_N];
8 } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13

```

```

13     for (int u = 1; u <= n; u++)
14         for (int v = 1; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     //if (u != v)
21     //    pG->A[v][u] += 1;
22
23     if (pG->A[u][v] > 1)
24         printf("da cung (%d, %d)\n", u, v);
25     if (u == v)
26         printf("khuyen %d\n", u);
27
28
29     pG->m++;
30 }
31
32 int adjacent(Graph *pG, int u, int v) {
33     return pG->A[u][v] > 0;
34 }
35
36 #define WHITE 0
37 #define GRAY 1
38 #define BLACK 2
39
40 int color[MAX_N];        //Lưu trạng thái của các đỉnh
41 int parent[MAX_N];       //Lưu trạng thái của các đỉnh
42 int has_circle;          //Đồ thị chứa trình hay không
43 int start, end;
44
45 void DFS(Graph *pG, int u, int p) {
46     //1. Tô màu đang duyệt cho u
47     color[u] = GRAY;
48     parent[u] = p;
49
50     //2. Xét các đỉnh kề của u
51     for (int v = 1; v <= pG->n; v++)
52         if (adjacent(pG, u, v)) {
53             if (color[v] == WHITE) //2a. Nếu v chưa duyệt
54                 DFS(pG, v, u);     //gọi đệ quy duyệt nó
55             else if (color[v] == GRAY) { //2b. v đang duyệt
56                 has_circle = 1;      //chứa chu trình
57                 start = u;
58                 end = v;
59             }
60         }
61
62     //3. Tô màu duyệt xong cho u
63     color[u] = BLACK;
64 }
65
66
67 int main() {
68     //1. Khai báo đồ thị G
69     Graph G;
70     //2. Đọc dữ liệu và dựng đồ thị
71     int n, m, u, v;
72     scanf("%d%d", &n, &m);
73     init_graph(&G, n);
74     for (int e = 0; e < m; e++) {
75         scanf("%d%d", &u, &v);
76         add_edge(&G, u, v);
77     }
78
79     for (int u = 1; u <= G.n; u++)
80         color[u] = WHITE;
81     //2. Khởi tạo biến has_circle
82     has_circle = 0;
83     //3. Duyệt toàn bộ đồ thị để kiểm tra chu trình
84     for (int u = 1; u <= G.n; u++)
85         if (color[u] == WHITE) //u chưa duyệt
86             DFS(&G, u, -1);    //gọi DFS(&G, u) để duyệt từ u
87     //4. Kiểm tra has_circle
88
89     if (has_circle) {
90         int A[100], i = 0;
91         //5.

```

```

91     A[0] = start;
92     int u = start;
93     do {
94         u = parent[u];
95         i++;
96         A[i] = u;
97     } while (u != v);
98
99     for (int j = i; j >= 0; j--)
100         printf("%d ", A[j]);
101
102     printf("%d\n", A[i]);
103 } else
104     printf("-1\n");
105
106 return 0;
107 }
108
109

```

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Viết chương trình đọc vào một **đơn đồ thị vô hướng** và kiểm tra xem nó có chứa chu trình hay không.

Nhắc lại: *Chu trình là một đường đi có đỉnh đầu trùng với đỉnh cuối.*

Đầu vào (Input):

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m, tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u, v mô tả cung (u, v).

Đầu ra (Output):

- In ra màn hình **CIRCLED** nếu đồ thị có chứa chu trình, ngược lại in ra **NO CIRCLE**
- Xem thêm ví dụ bên dưới.

For example:

Input	Result
3 2 1 3 3 2	NO CIRCLE
7 9 1 2 1 3 1 4 2 3 2 6 3 7 4 5 5 3 5 7	CIRCLED
7 11 1 2 1 3 2 4 2 5 2 6 3 2 3 5 3 6 4 7 5 7 6 4	CIRCLED
3 2 1 2 2 3	NO CIRCLE
4 3 1 2 2 3 4 2	NO CIRCLE

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4 typedef int ElementType;
5
6 //-----Graph-----
7 typedef struct{
8     int n,m;
9     int A[MAX_SIZE][MAX_SIZE];
10 }Graph;
11
12 //init graph with n vertex
13 void init_graph(Graph* pG, ElementType n){
14     pG->n = n;
15     pG->m = 0;
16
17     //cho toàn bộ ma trận ko hàng 0
```

```
17 //Cho toàn bộ ma trận Ké bằng 0
18 for(int i = 0; i <= pG->n; i++){
19     for(int j = 0; j <= pG->n; j++){
20         pG->A[i][j] = 0;
21     }
22 }
```

Debug: source code from all test runs

Run 1




```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v]++;
    if(u!=v){
        pG->A[v][u]++;
    }
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s, int p){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(p==u) continue;
            if(color[u] == WHITE){
                DFS(pG,u,s);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);

```

```
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
has_cycle = 0;
for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
    parent[i] = -1;
    color[i] = WHITE;
}

for(int i = 1; i <= G.n; i++){
    if(!visited[i]){
        DFS(&G,i,-1);
    }
}

if(has_cycle){
    printf("CIRCLED");
}else{
    printf("NO CIRCLE");
}
}
```

Run 2

```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v]++;
    if(u!=v){
        pG->A[v][u]++;
    }
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s, int p){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(p==u) continue;
            if(color[u] == WHITE){
                DFS(pG,u,s);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);

```

```
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
has_cycle = 0;
for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
    parent[i] = -1;
    color[i] = WHITE;
}

for(int i = 1; i <= G.n; i++){
    if(!visited[i]){
        DFS(&G,i,-1);
    }
}

if(has_cycle){
    printf("CIRCLED");
}else{
    printf("NO CIRCLE");
}
}
```

Run 3

```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v]++;
    if(u!=v){
        pG->A[v][u]++;
    }
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s, int p){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(p==u) continue;
            if(color[u] == WHITE){
                DFS(pG,u,s);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);

```

```
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
has_cycle = 0;
for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
    parent[i] = -1;
    color[i] = WHITE;
}

for(int i = 1; i <= G.n; i++){
    if(!visited[i]){
        DFS(&G,i,-1);
    }
}

if(has_cycle){
    printf("CIRCLED");
}else{
    printf("NO CIRCLE");
}
}
```

Run 4

```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v]++;
    if(u!=v){
        pG->A[v][u]++;
    }
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s, int p){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(p==u) continue;
            if(color[u] == WHITE){
                DFS(pG,u,s);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);

```

```
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
has_cycle = 0;
for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
    parent[i] = -1;
    color[i] = WHITE;
}

for(int i = 1; i <= G.n; i++){
    if(!visited[i]){
        DFS(&G,i,-1);
    }
}

if(has_cycle){
    printf("CIRCLED");
}else{
    printf("NO CIRCLE");
}
}
```

Run 5


```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v]++;
    if(u!=v){
        pG->A[v][u]++;
    }
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s, int p){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(p==u) continue;
            if(color[u] == WHITE){
                DFS(pG,u,s);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);

```

```
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
has_cycle = 0;
for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
    parent[i] = -1;
    color[i] = WHITE;
}

for(int i = 1; i <= G.n; i++){
    if(!visited[i]){
        DFS(&G,i,-1);
    }
}

if(has_cycle){
    printf("CIRCLED");
}else{
    printf("NO CIRCLE");
}
}
```

Run 6

```

#include <stdio.h>

#define MAX_SIZE 100
typedef int ElementType;

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

//init graph with n vertex
void init_graph(Graph* pG, ElementType n){
    pG->n = n;
    pG->m = 0;

    //cho toan bo ma tran ke bang 0
    for(int i = 0; i <= pG->n; i++){
        for(int j = 0; j <= pG->n; j++){
            pG->A[i][j] = 0;
        }
    }
}

//add edge (u,v) to graph
void add_edge(Graph* pG, ElementType u, ElementType v){
    pG->A[u][v]++;
    if(u!=v){
        pG->A[v][u]++;
    }
    pG->m++;
}

//-----DFS-----
#define WHITE 0
#define GRAY 1
#define BLACK 2

int color[MAX_SIZE];
int has_cycle;

int visited[MAX_SIZE];
int parent[MAX_SIZE];

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

void DFS (Graph* pG, int s, int p){
    //1. to mau
    color[s] = GRAY;

    //    visited[s] = 1;
    //    printf("%d \n", s);
    parent[s] = p;
    for(int u = 1; u <= pG->n; u++){
        if(adj(pG,s,u)){
            if(p==u) continue;
            if(color[u] == WHITE){
                DFS(pG,u,s);
            } else if(color[u] == GRAY){
                has_cycle = 1;
            }
        }
    }
    color[s] = BLACK;
}

int main(){
    Graph G;
    int n, m, u, v, e;
    scanf("%d%d", &n, &m);

```

```
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
has_cycle = 0;
for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
    parent[i] = -1;
    color[i] = WHITE;
}

for(int i = 1; i <= G.n; i++){
    if(!visited[i]){
        DFS(&G,i,-1);
    }
}

if(has_cycle){
    printf("CIRCLED");
}else{
    printf("NO CIRCLE");
}
}
```

	Input	Expected	Got	
✓	3 2 1 3 3 2	NO CIRCLE	NO CIRCLE	✓
✓	7 9 1 2 1 3 1 4 2 3 2 6 3 7 4 5 5 3 5 7	CIRCLED	CIRCLED	✓
✓	7 11 1 2 1 3 2 4 2 5 2 6 3 2 3 5 3 6 4 7 5 7 6 4	CIRCLED	CIRCLED	✓
✓	3 2 1 2 2 3	NO CIRCLE	NO CIRCLE	✓
✓	4 3 1 2 2 3 4 2	NO CIRCLE	NO CIRCLE	✓
✓	4 4 1 2 2 3 4 3 4 1	CIRCLED	CIRCLED	✓

Question author's solution (C):

```
1 #include <stdio.h>
2
3 /* Khai báo CTDL Graph*/
4 #define MAX_N 100
5 typedef struct {
6     int n, m;
7     int A[MAX_N][MAX_N];
8 } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     if (u != v)
21         pG->A[v][u] += 1;
22 }
```

Correct

Marks for this submission: 1.00/1.00.

Viết chương trình đọc vào một **đơn đồ thị vô hướng** và kiểm tra xem nó có chứa chu trình hay không. Nếu đồ thị có chu trình, in ra các đỉnh có trong chu trình.

Đầu vào (Input):

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u, v mô tả cung (u, v) .

Đầu ra (Output):

- Nếu đồ thị có chu trình, in ra các đỉnh có trong chu trình trên cùng 1 dòng, cách nhau 1 khoảng trắng. Đỉnh đầu và đỉnh cuối của chu trình giống nhau.
- Nếu đồ thị không chứa chu trình, in ra -1
- Nếu đồ thị có nhiều chu trình, chỉ cần in ra 1 chu trình bất kỳ.
- Xem thêm ví dụ bên dưới.

Trong ví dụ đầu tiên, ta tìm thấy chu trình $1 \rightarrow 3 \rightarrow 2 \rightarrow 1$, vì thế chương trình phải in ra:

1 3 2 1

For example:

Input	Result
3 3 1 3 2 3 1 2	Chu trình hợp lệ.
7 9 1 2 1 3 1 4 2 3 2 6 3 7 4 5 5 3 5 7	Chu trình hợp lệ.

Answer: (penalty regime: 10, 20, ... %)

```

1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4  typedef int ElementType;
5
6  //-----Graph-----
7  typedef struct {
8      int n, m;
9      int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 void init_graph(Graph* pG, int n) {
13     pG->n = n;
14     pG->m = 0;
15     for (int i = 0; i <= n; i++) {
16         for (int j = 0; j <= n; j++) {
17             pG->A[i][j] = 0;
18         }
19     }
20 }
21
22 void add_edge(Graph* pG, int u, int v) {
23     pG->A[u][v]++;
24     if (u != v) pG->A[v][u]++;
25
26     pG->m++;
27 }
28
29 int adj(Graph* pG, int u, int v) {
30     return pG->A[u][v] > 0;
31 }
```

```

31 }
32 //-----Cycle-----
33 #define NO_COLOR 0
34 #define GRAY 1
35 #define BLACK 2
36
37 int color[MAX_SIZE];
38 int has_cycle;
39 int parent[MAX_SIZE];
40
41 int start, end;
42
43 void cycle(Graph* pG, int u, int p) {
44     color[u] = GRAY;
45
46     for (int v = 1; v <= pG->n; v++) {
47         if (adj(pG, u, v)) {
48             if (p==v) continue;
49             if (color[v] == NO_COLOR) {
50                 parent[v] = u;
51                 cycle(pG, v, u);
52                 if (has_cycle) return;
53             } else if (color[v] == GRAY) {
54                 has_cycle = 1;
55                 start = v;
56                 end = u;
57                 return;
58             }
59         }
60     }
61     color[u] = BLACK;
62 }
63
64 void print_path() {
65     int path[MAX_SIZE];
66     int count = 0;
67
68     path[count++] = start;
69     int u = end;
70     while (u != start) {
71         path[count++] = u;
72         u = parent[u];
73     }
74     path[count++] = start;
75
76     for (int i = count - 1; i >= 0; i--) {
77         printf("%d ", path[i]);
78     }
79     printf("\n");
80 }
81
82 int main() {
83     int n, m;
84     scanf("%d %d", &n, &m);
85
86     Graph G;
87     init_graph(&G, n);
88
89     // Read graph
90     for (int i = 0; i < m; i++) {
91         int u, v;
92         scanf("%d %d", &u, &v);
93         add_edge(&G, u, v);
94     }
95
96     for (int i = 1; i <= n; i++) {
97         color[i] = NO_COLOR;
98         parent[i] = -1;
99     }
100
101     for (int i = 1; i <= G.n; i++) {
102         if (color[i] == NO_COLOR) {
103             start = i;
104             cycle(&G, i, -1);
105             if (has_cycle) break;
106         }
107     }
108
109     if (has_cycle) {

```

```

109 11 (has_cycle) {
110     print_path();
111 } else {
112     printf("-1");
113 }
114 // for (int i = 1; i <= G.n; i++) {
115 //     printf("%d: %d\n", i, parent[i]);
116 // }
117 }
118

```

	Input	Expected	Got	
✓	3 3 1 3 2 3 1 2	Chu trình hợp lệ.	Chu trình hợp lệ.	✓
✓	7 9 1 2 1 3 1 4 2 3 2 6 3 7 4 5 5 3 5 7	Chu trình hợp lệ.	Chu trình hợp lệ.	✓
✓	7 11 1 2 1 3 2 4 2 5 2 6 3 2 5 3 3 6 4 7 7 5 6 4	Chu trình hợp lệ.	Chu trình hợp lệ.	✓
✓	3 2 1 2 2 3	-1	-1	✓
✓	4 3 1 2 2 3 4 2	-1	-1	✓
✓	4 4 1 2 2 3 3 4 4 1	Chu trình hợp lệ.	Chu trình hợp lệ.	✓

Passed all tests! ✓

Question author's solution (Python3):

```

1 #include <stdio.h>
2
3 /* Khai báo CTDL Graph*/
4 #define MAX_N 100
5 typedef struct {
6     int n, m;
7     int A[MAX_N][MAX_N];
8 } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;

```



```

12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     if (u != v)
21         pG->A[v][u] += 1;
22
23     if (pG->A[u][v] > 1)
24         printf("da cung (%d, %d)\n", u, v);
25     if (u == v)
26         printf("khuyen %d\n", u);
27
28
29     pG->m++;
30 }
31
32 int adjacent(Graph *pG, int u, int v) {
33     return pG->A[u][v] > 0;
34 }
35
36 #define WHITE 0
37 #define GRAY 1
38 #define BLACK 2
39
40 int color[MAX_N];          //Lưu trạng thái của các đỉnh
41 int parent[MAX_N];        //Lưu trạng thái của các đỉnh
42 int has_circle;           //Đồ thị chứa trình hay không
43 int start, end;
44
45 void DFS(Graph *pG, int u, int p) {
46     //1. Tô màu đang duyệt cho u
47     color[u] = GRAY;
48     parent[u] = p;
49
50     //2. Xét các đỉnh kề của u
51     for (int v = 1; v <= pG->n; v++)
52         if (adjacent(pG, u, v)) {
53             if (v == p)
54                 continue;
55
56             if (color[v] == WHITE) //2a. Nếu v chưa duyệt
57                 DFS(pG, v, u);     //gọi đệ quy duyệt nó
58             else if (color[v] == GRAY) { //2b. v đang duyệt
59                 has_circle = 1;     //chứa chu trình
60                 start = u;
61                 end = v;
62             }
63         }
64
65     //3. Tô màu duyệt xong cho u
66     color[u] = BLACK;
67 }
68
69
70 int main() {
71     //1. Khai báo đồ thị G
72     Graph G;
73     //2. Đọc dữ liệu và dựng đồ thị
74     int n, m, u, v;
75     scanf("%d%d", &n, &m);
76     init_graph(&G, n);
77     for (int e = 0; e < m; e++) {
78         scanf("%d%d", &u, &v);
79         add_edge(&G, u, v);
80     }
81
82     for (int u = 1; u <= G.n; u++)
83         color[u] = WHITE;
84     //2. Khởi tạo biến has_circle
85     has_circle = 0;
86     //3. Duyệt toàn bộ đồ thị để kiểm tra chu trình
87     for (int u = 1; u <= G.n; u++)
88         if (color[u] == WHITE) //u chưa duyệt
89             DFS(&G, u, -1);    //gọi DFS(&G, u) để duyệt từ u

```

```

90 //4. Kiểm tra has_circle
91
92 if (has_circle) {
93     int A[100], i = 0;
94     A[0] = start;
95     int u = start;
96     do {
97         u = parent[u];
98         i++;
99         A[i] = u;
100     } while (u != v);
101
102     for (int j = i; j >= 0; j--)
103         printf("%d ", A[j]);
104
105     printf("%d\n", A[i]);
106 } else
107     printf("-1\n");
108
109 return 0;
110 }
111
112

```

Correct

Marks for this submission: 1.00/1.00.

◀ * Bài tập 9. Ứng dụng liên thông

Jump to...

* Bài tập 11 - Ứng dụng kiểm tra chu trình ▶

